AI

MDPI

Article

Using a High-Precision YOLO Surveillance System for Gun Detection to Prevent Mass Shootings

Jonathan Hsueh [1] and Chao-Tung Yang [2,3,4,*]

[1] Department of Computer Science, University of California Santa Barbara, Santa Barbara, CA 93106, USA; jonathanhsueh@ucsb.edu
[2] Department of Computer Science, Tunghai University, Taichung City 407224, Taiwan
[3] Research Center for Smart Sustainable Circular Economy, Tunghai University, Taichung City 407224, Taiwan
[4] Department of Medical Research, Kuang Tien General Hospital, Taichung 433004, Taiwan
* Correspondence: ctyang@thu.edu.tw

Abstract

Mass shootings are forms of loosely defined violent crimes typically involving four or more casualties by firearm and have become increasingly more frequent, and organized and speedy responses from police are necessary to mitigate harm and neutralize the perpetrator. Recent, widely publicized police responses to mass shooting events have been criticized by the media, government, and public. With the advancements in artificial intelligence, specifically single-shot detection (SSD) models, computer programs can detect harmful weapons within efficient time frames. We utilized YOLO (You Only Look Once), an SSD with a Convolutional Neural Network, and used versions 5, 7, 8, 9, 10, and 11 to develop our detection system. For our data, we used a Roboflow dataset that contained almost 17,000 images of real-life handgun scenarios, designed to skew towards positive instances. We trained each model on our dataset and exchanged different hyperparameters, conducting a randomized trial. Finally, we evaluated the performance based on precision metrics. Using a Python-based design, we tested our model's capabilities for surveillance functions. Our experimental results showed that our best-performing model was YOLOv10s, with an mAP-50 (mean average precision 50) of 98.2% on our dataset. Our model showed potential in edge computing settings.

Keywords: YOLO; gun detection; surveillance; edge computing; image detection

check for updates

Academic Editor: Miguel Angel Cazorla

Received: 12 July 2025
Revised: 13 August 2025
Accepted: 20 August 2025
Published: 22 August 2025

Citation: Hsueh, J.; Yang, C.-T. Using a High-Precision YOLO Surveillance System for Gun Detection to Prevent Mass Shootings. *AI* **2025**, *6*, 198. https://doi.org/10.3390/ai6090198

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In the modern lexicon, mass shootings are forms of violent crimes that result in four or more deaths/casualties due to firearms [1]. They have become increasingly more frequent as of the 21st century.

Significant Mass Shooting Events of the 20–21st Century:

- The Virginia Tech Shooting, in the United States, resulted in 32 deaths and 23 injuries. Perpetrated with two semi-automatic pistols, it remains the deadliest school shooting event in American History. The shooter had years of severe, neglected behavioral problems.
- The Bacha Khan University Attack, near Charsadda, Pakistan, resulted in the deaths of 22 people and the injuries of 20 more. Armed militants often target schools for terror attacks, and assault rifles and grenades were the weapons of choice.

- The Kerch Polytechnic College Massacre, in Kerch, Crimea, was one of the deadliest school shootings of the 21st century in Russia. Perpetrated by a lone gunman who was an academically struggling student, the event resulted in the deaths of 21 students and the injuries of 67.
- The Rio de Janeiro Elementary School Shooting, in Rio de Janeiro, Brazil, resulted in the deaths of 12 students and over 20 serious injuries. The perpetrator acted alone with two revolvers and was described as a reserved person, who was a victim of bullying and an obsessive about terrorist acts.

According to the world literature, mass shooting events specifically in locations of education differ from continent to continent. In the Americas and Europe, lone gunmen perpetrate solo acts of indiscriminate killing for retaliatory or psychopathic reasons [2]. Common traits include having neglected behavior issues, struggles with bullying, and access to personal defense weapons [2]. In places with contested borders, such as Crimea, mass shooting events often trigger political issues, despite investigations revealing lone perpetrators [3]. In some parts of Africa and Asia, places of education are targeted by armed militants and terror groups for religious or political reasons. They are armed with a diverse array of weapons that require systemic changes to ensure security [4]. For this paper, we focus on stopping lone-shooter events as they have greater potential for research prevention.

With the advancement of technology, artificial intelligence has evolved to where image recognition has found extensive applications, especially in security [5,6]. Manufacturing ability has progressed to where components can be made smaller and more power-efficient [7]. The algorithm evolution has achieved more high-speed, accurate, and energy-saving solutions to solving problems. Additionally, differences in AI architecture have allowed specialization in different fields [8]. With these abilities, our work uses YOLO models as both the main development environment and primary testing/training algorithm.

Focusing on the topic of our paper, AI has found applications in security settings [9]. Weapon detection systems can quickly identify guns and illegal firearms, improving police response time to prevent serious bodily harm [10]. In this field of study, many decisions and architectural choices have to be made to optimize for the given task. Security detection models have to be lightweight and portable to suit a diverse range of immediate, real-time needs [10]. Common models such as Faster-Rcnn with Resnet-101 and VGG16 backbones require large amounts of space (500 to 1000 MB) and vast computational resources (3–12 RAM GB) [11]. In response, we hypothesize that using lightweight models would give the most benefit. They would retain small-object-detection ability and efficient single-shot class identification. YOLO models are computationally efficient, lightweight applications [12]. We also realized that the most important tasks required identifying misshapen, augmented firearm images that might escape security personnel's focus. We theorize that using images of handguns, the most common firearm in violent crime, would be the most efficient at preventing overfitting. Lastly, we plan to make a key design change in heavily skewing our dataset towards true-positive instances, promoting high precision of any firearm instance at the potential drawback of "false alarms." Our research observes the Consort-AI reporting guidelines in outlining our AI intervention's purpose, for the purpose of conducting a randomized, controlled trial involving AI. Our main research objectives are as follows:

- To develop object detection technology into a weapon detection system that can identify harmful firearms in security or large environment settings, optimized for high precision.
- To compare widely utilized object detection algorithms, which include YOLOv5, YOLOv7, YOLOv8, YOLOv9, YOLOv10, and YOLOv11.

- To test and implement object detection for a firearm detection surveillance system accessible to diverse device inputs, intended for usage among security personnel.

2. Prelimary Works

This section provides the background of this work, including related works and information. Similar research has been conducted in machine learning, neural networks, and deep learning algorithms such as YOLO. Convolutional Neural Networks have also seen rapid development.

We conducted a literature review on Google Scholar with the query "Artificial Intelligence in Handgun Detection," identifying over 100 relevant papers. Applying exclusion criteria (such as being relevant to AI as well as object and weapon detection) gave us a final result of around 60 papers. Our review processed analyzed research data to inform our decision-making. For architecture design, we gathered the model type, backbone choices, and design changes. We gathered hyperparameter information such as training epochs and augmentations. We analyzed which combinations of model design and parameters produced the highest-precision training results, focusing on tradeoffs of more lightweight backbones, higher epochs, and single-shot detectors. We used performance metric data such as accuracy and mAP50 to benchmark our reviewed models for precision capabilities. We also used the detection speed to assess suitability in real-life gun detection scenarios. Any significant result would be a design that had high-precision and fast inference qualities. Finally, we focused on the dataset design including the type of images, image count, and class distribution. This allowed us to identify research gaps specific to gun detection, specifically the lack of rifled firearm images.

We used research from established journals such as Elsevier's *IoT*, ScienceDirect's *Machine Learning with Applications*, MDPI's *AI*, and Springer's *Artificial Intelligence Review*. In summary, our methodological approach to the literature review maintained a systematic and rigorous process, with careful paper selection and data analysis in line with academic standards and research objectives.

2.1. Related Works

Ahmed and Echi tested Hawk-Eye, an AI-powered threat detector, to test its weapon detection capabilities [11]. They proposed a system to have a camera side and a cloud side. The camera side had a CNN that was trained with four convolutional layers, with the goal of learning a compressed representation of the dataset. The cloud side had a pre-trained Mask R-CNN model to construct segmentation masks of weapons on the images. For their dataset, they used public ones on Kaggle, and they collected a total of over 10 thousand images of weapons in their natural environments. The model had accuracies of 92% and 95% for pistols and knives, respectively. It had a 100% accuracy in detecting machine guns, one of the most common weapons in the deadliest mass shootings; therefore, it has potential usage in America.

Kieran et al. proposed a model with an SSD and Faster RCNN algorithms [13]. These models are enhanced to increase efficiency and reduce costs [14]. Faster RCNN, although modified and improved, has not been fast enough to be used in real-life settings. They also make use of YOLO, and the SSD and CNN design make it usable in real time. Their data came from pre-labeled video datasets, so the model could have real-time security footage applications. The Faster RCNN algorithm took 1.606 s and had an accuracy of 84.6%. YOLO, the SSD, took 0.736 s and had an accuracy of 73.8%.

Xu and Hung made use of a wide range of artificial intelligence models [15]. They first test and discuss the Athena Security AI System and its potential in weapon detection. Then, they used a COCO dataset of 1218 machine gun images. These were processed from

RGB to grayscale, and then the researchers sent the processed images to an SSD-MobileNet model, which consisted of a Single-Shot MultiBox Detector and a MobileNet lightweight deep neural network. The MobileNet contains a Convolutional Neural Network that performs feature extraction at different scales. They utilize key frame extraction and weapon detection. At the intersection over union (IoU) value of 0.5 to 0.75, the system achieved an accuracy of 0.8524 and 0.7006, respectively.

The authors of this paper utilized YOLOv7 and YOLOv8 artificial intelligence models [16]. Their training and testing dataset was provided in the YOLO format, which was the corresponding txt file. When evaluating the model's performance, they focused on the precision and mean average precision metrics. YOLOv8 was the fastest model, while YOLOv7-e6 had the longest training time but the best results. Analyzing the results, the model's performance stabilized around 90 epochs, indicating decreased learning correlated to increased epochs. They trained different versions of YOLOv7 and the nano version of YOLOv8 for 100 epochs. Their times were around an hour. Overall, YOLOv7-e6 had the highest results, with a precision of 0.911, a recall of 0.837, and an mAP50 of 0.903.

The authors of this paper make use of a wide variety of single and two-shot object detectors, and they end up focusing on the single-shot detector YOLO [17]. YOLOv8 was their version of choice. For data, the authors created a 9633-image dataset with the classes pistol, missile, gun, grenade, and knife. They meticulously annotated each image with bounding boxes. They applied necessary cleaning and augmentation to improve the model's generalization ability. During their training phase, the model ran the dataset through its neural network. They ran YOLOv8 from a range of 25–40 epochs to further fine-tune the model. Finally, they tested the improved dataset and proposed factors to increase model efficiency, such as utilizing quantization techniques and optimizing hyperparameters. YOLOv8 had a final detection rate of 48.6% mAP at the 50% confidence level. The model showed great performance in detecting concealed weaponry.

Jadhav et al. focus on manual surveillance issues such as human error and propose machine intelligence solutions to address them [18]. They survey deep learning object detection models such as Faster RCNN, RFCNN, SSDs, and YOLO, and they implement CNN, VGG, YOLOv7, and YOLOv8. Their single-class dataset consisted of 2971 images of guns. Their methodology includes dataset preparation, input identification, database storage, filter application, and alarm system installation. Their results showed that YOLOv8 outperformed the rest of the models with an F1-score of 89.1% and an mAP50 of 87.7%.

The authors of this paper seek to meet military surveillance needs with YOLO algorithms [19]. They compare YOLO versions 3, 4, and 5m (medium) on a publicly available dataset for military object detection. They add person, gun, knife, and drone classes to the dataset to improve viability in military settings. They tackle small-object detection challenges by integrating the Slicing Aided Hyper Inference (SAHI) with the YOLOv5m model. The SAHI algorithm employs a slicing-assisted cropping technique to improve the detection of small-object instances. Their dataset contains six classes and 6772 images. During the training process, YOLOv5m stopped at 90 epochs due to a lack of training loss reduction. YOLOv3 and YOLOv4 were both trained to 12000 epochs. They encountered problems in their Military Image Object Detection (MIOD) dataset, including annotation inconsistency among person and gun classes, and they resolved to manually label the images, which increased their mAP50 value. YOLOv5m achieved the best mAP50 on their first dataset with a score of 95.9%. It also achieved the greatest mAP50 on the second dataset with a score of 82.3%. The authors attributed much of this success to the powerful slicing-aided hyperinference technique to improve small-object detection.

2.2. Virtual Signal Large Model

The authors of this paper suggest a Virtual Signal Large Model (VSLM) in few-shot and close-domain scenarios to tackle the Wideband Signal Detection and Recognition method shortcomings of requiring large, well-labeled datasets [20]. They designed two plug-and-play modules, their virtual sample generations (VSGs) and virtual category generations (VCGs), respectively, which allow the program to adapt to hardware changes. They proposed a Dual Decoupled Network to train the VSLM, which could refer to separating ML training into separate sections of feature extraction and prediction. This method improves signal details by decoupling low gray values, which alleviates conflicts during joint optimizations involving multiple tasks. This modularized approach to machine learning optimizes efficiency at every level of the learning and data analysis process. They achieved 98% average precision 50 on their datasets.

2.3. Research Gaps and Literature Contributions

After completing our review, we identified the research gaps of the absence of newer YOLO models (including versions 9, 10, and 11) and different training techniques, such as transfer and deep learning. The majority of papers tested YOLO models 3–8 and utilized supervised learning. We focus on single-shot detectors [21]. This gap indicates a lack of research on newer machine learning models (YOLO) and data testing techniques. Another issue is that many papers lack a design for a surveillance-based weapon-detection system for their technology, which translates into issues of lacking real-world effects and a primary motive/impact [22]. Most introductions had unclear motives and vague focuses within the gun detection field, which could include being motivated by recent gun violence and targeting organized crime. A concise summary containing surveyed methods with their advantages and disadvantages can be found in Table 1.

Table 1. Surveyed weapon detection strategies.

Strategy	Description	Advantages	Disadvantages
Images	Contained actual events	realistic data	difficult to source
Design	Balanced True Pos/Neg	well-rounded	lower precision
RCNN	Resnet-101 or VGG16	high accuracy	>latency and computations
YOLO	SSD, one forward pass	high mAP and speed	complexity struggles

We decide to focus on newer YOLO versions including 9, 10, and 11. Each of those three YOLO models were designed with enhanced feature extraction, accuracy across fewer parameters, and speed latency. Next, we focus on enhancing handgun detection (prevention of mass shootings specifically on college campuses) due to the relative prevalence of related images and the frequency of handguns in violent crime. Finally, we optimize for precision in security settings.

3. Method and System Implementation

Figure 1 demonstrates our work's architecture from beginning to end. Our model has four parts including image inputting, client training, result gathering, and remote monitoring/hosting.

First, we gather real-life images involving firearms, which comes from publicly available sources such as Roboflow and Kaggle since acquiring firearms is costly. Next, we train YOLO models with this data to improve firearm detection capabilities. We apply different augments and training methods. After, we analyze the results to understand our model's performance and optimize future trainings. Finally, we create and propose solutions to establish a user client using the YOLO weights, where this model can be accessible to any internet device.

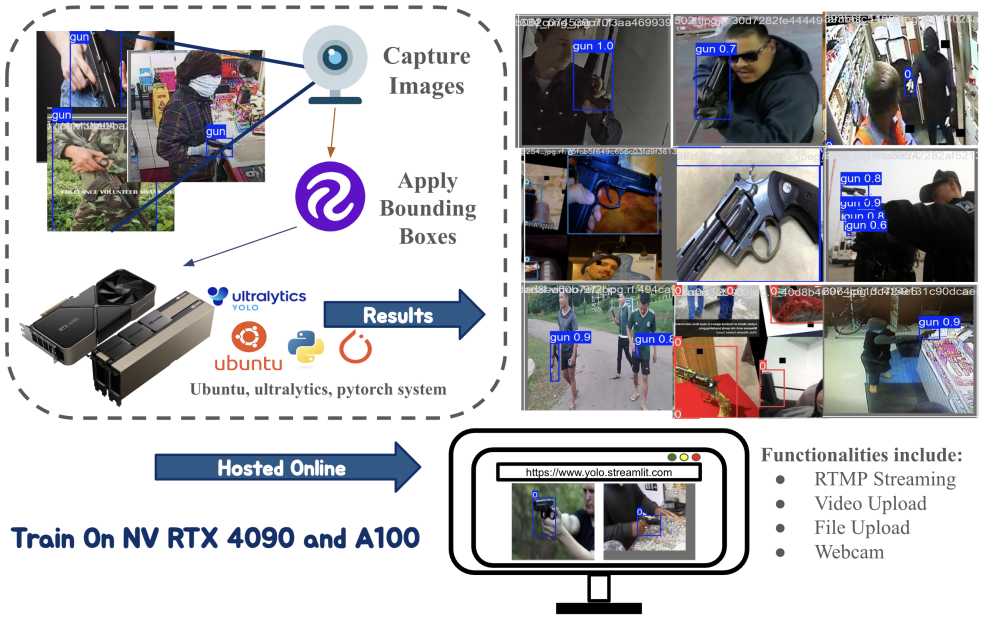


Figure 1. System workflow diagram.

3.1. You Only Look Once: Unified, Real-Time Object Detection

For real-time applications, we utilize a single-shot detector model You Only Look Once (YOLO) that uses a single convolutional network to detect objects, which differs from two-stage detectors such as Faster R-CNN that use regional proposal networks [23]. Our main goal was speed. We tested versions 5, 7, 8, 9, 10, and 11, optimizing for object detection, image classification, and object localization [24–30].

A brief history of the changes in YOLO architecture is as follows:

- YOLOv7 is 120% faster than YOLOv5, trained solely on the MS COCO dataset. A key innovation was a proposed extended ELAN (Efficient Layer Aggregation Networks).
- YOLOv8 is 22% faster than YOLOv7 in achieving 0.5 mAP50:95 on the COCO dataset. It incorporates an anchor-free split Ultralytics head and an advanced CSPDarknet53 backbone, using Cross-Stage Partial connection.
- YOLOv9 achieves on average 2–3% more mAP50:95 than YOLOv8 every 10 million parameters. It uses key innovations such as Programmable Gradient Information (PGI), Generalized Efficient Layer Aggregation Network (GELAN), and reversable functions, to name a few.
- YOLOv10 is 14% faster than YOLOv9 on the COCO dataset and is built off the ultralytics package. It removes non-maximum supression, replacing it with consistent dual-assignments, and has an advanced version of CSPNet (Cross Stage Partial Network) to improve gradient flow.
- YOLOv11 has 22% fewer parameters than YOLOv8m (while achieving greater mAP) and is 7.4% faster than YOLOv10 on the COCO dataset. The model employs an improved backbone and neck architecture and is supported on a broad range of tasks including edge devices.

Single-shot detectors (SSDs) are a lightweight ML design that uses Convolutional Neural Networks (CNNs) to make detections in a single shot [31]. They mimic the design of natural brain neurons to abstract an image into a familiarity detection [32]. An example of the design architecture can be found in Figure 2.

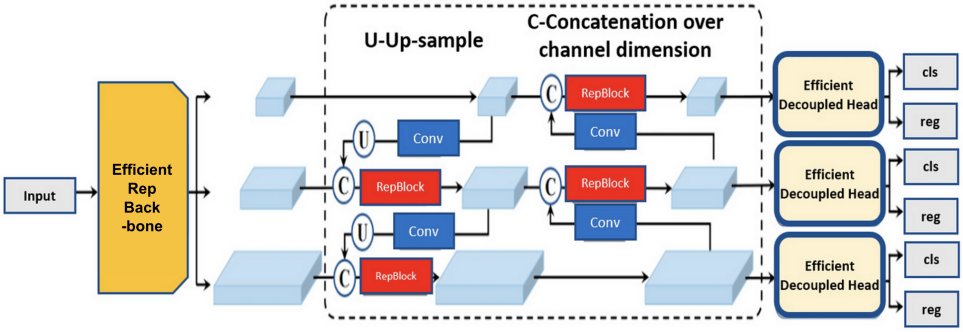


Figure 2. YOLOv8 design architecture.

With SSDs, their initial shot is made up of a set containing grids, each of which is responsible for detecting objects, and each cell contains predefined/pretrained anchor boxes that help with finding multiple objects in a singular grid cell (1). The model vectorizes data points, turning data such as images, pixels, and words into location vectors called tensors that the model excells at. They are added to a feature map with the dimensions in Equation (2), represented by Figure 3.

Image Tensor Dimensions:

$$(num\ inputs) \times (input\ height) \times (input\ width) \times (input\ channel) \tag{1}$$

Feature Map Dimensions:

$$(num\ inputs) \times (map\ height) \times (map\ width) \times (map\ channel) \tag{2}$$

Here are specific details of our AI algorithm to align with Consort-AI guidelines. Our system uses a **self-organizing feature map** (SOFM) to extract unique identifiers from inputted data and help to visualize higher-order datasets. It represents a dataset with p variables measured in n observations into a cluster of observations, creating two-dimensional projections of higher-order datasets. The YOLO-trained object tensors represent nodes and neurons with a weight vector, and our training moves the vectors towards the inputted data through reducing the euclidean distance with Equation (3).

$$d(p, q) = \sqrt{(p_1, q_1)^2 + (p_2, q_2)^2} \tag{3}$$

For our training, we use competitive learning, a type of unsupervised learning where the nodes compete for the right to respond to the inputted data, increasing the specialization of each node in the network. The competitive layer contains nodes described by a vector of weights $W_i = (w_i1, \ldots, w_id)^T$, $i = 1, \ldots, M$ and calculates a similarity measure between the inputted data $X^n = (x_n1, \ldots, x_nd)^T \in \mathbb{R}^d$ and the weight vector W_i. We find the most similar input vector by calculating the inverse Euclidean distance $\| x^n - W_i \|$ between the input vector x^n and the weight vector W_i. This avoids specious classification from misleading weight vectors, outlined in Equation (4).

Finding the neuron whose weight vector is most similar to the input data is the **Best-Matching Unit** (BMU). Formula with weight vector $W_v(s)$:

$$W_v(s + 1) = W_v(s) + \theta(u, v, s) \cdot \alpha(s) \cdot (D(t) - W_v(s)) \tag{4}$$

where

- s is the step index;
- t is the index to the training batch;
- u is the index of the BMU for the input vector $D(t)$;
- $\theta(u, v, s)$ is the neighborhood function that gives the distance between the neuron u and neuron v in step s.

We repeat this process for a large number of cycles λ, and the network brings together similar output nodes to form an input set. The SOFM forms a semantic mapping of items/objects that have excited adjacent neurons. Our YOLO model's maps represent learned characteristics and patterns in the inputted images. The SSD applies more convolutional layers to the "backbone" feature map. Stacked layers are sent to the model's "neck" for feature agregation and refinement. The final results are sent to the head for specific predictions. Our architecture ensures higher confidence scores and a tighter bounding boxs.

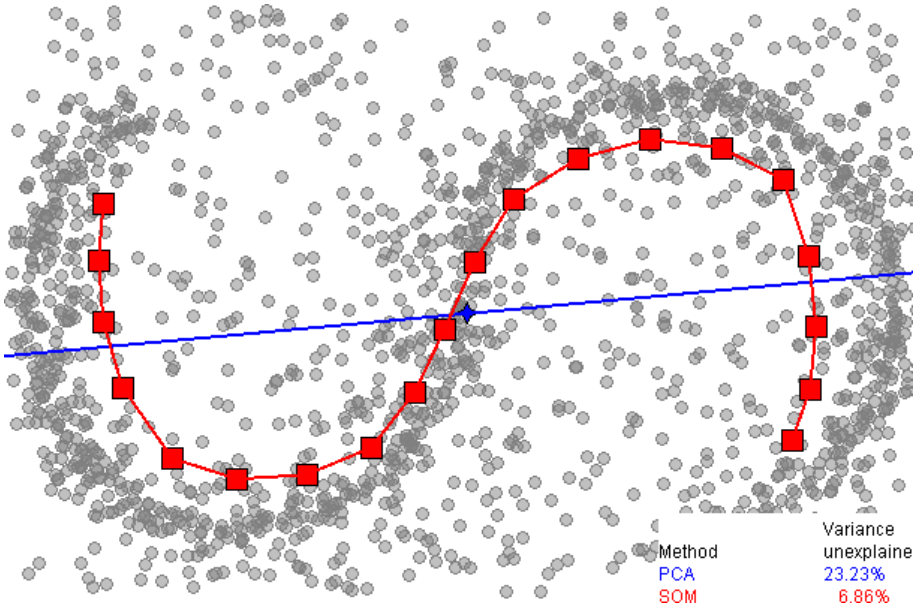


Figure 3. One- dimensional feature map.

Finally, the model uses **Non-Maximum Supression** to filter out bounding boxes and choose the one with the highest confidence score p_c. Each output prediction (5) contains the x and y coordinates and the box's height and width. The model chooses a cutoff value, for example, 0.5, and it discards all boxes $p_c \leq 0.5$ and picks the one with the largest IoU.

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix} \tag{5}$$

3.2. Dataset

Our dataset is publicly available on Roboflow [33]. Titled "CS 231N Project Computer Vision Project," it has 17,971 total images of mostly handguns, as well as other types of rifles, labeled with the singular class "guns" and bounding boxes [34]. This dataset has a great number of publically available, real-life scenario photos, such as officer bodycams or security camera footage. In gun instances, weapons are primarily being held by people with part of the barrel obscured. The image qualities were assessed for potentially malformed

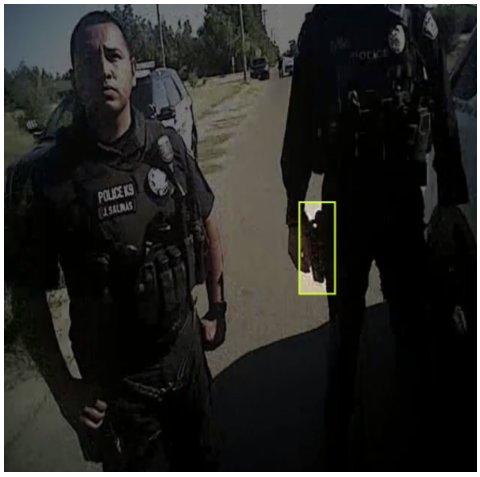
classes or bounding boxes. The original resolutions ranged from 1280 × 800 to 1600 × 1024, and each image was refitted to 640 × 640. Next, we applied the augmentations such as of removing select pixels, rotating the images, mosaic creating new contexts, and flipping to prevent overfitting (when the model performs poorly on slightly modified images) [35]. The split was 82%–14%–4% training–validation–testing. The split gave us the best results in preventing overfitting and hyperparameter tuning. While we considered using other splits such as cross-validation, we found it less compatible with the skew in our dataset. Our split was informed by the large number images and true-positive instances [36]. The design of our dataset informed much of performance metric and model architecture decisions. The features of our dataset can be found in Table 2. Refer to Figure 4 for view images in our dataset. All images utilized are publically available on Roboflow.

Table 2. Dataset class distribution.

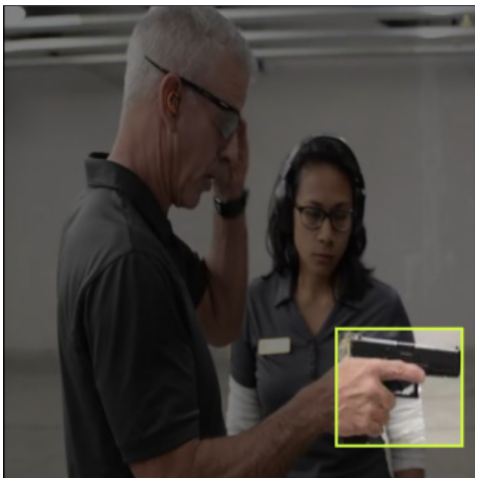
Classes	Images	Description	Split	Augmentations Dims
gun	17,826	security, bodycam	14,617-2496-713	mosaic, erasing
null	145	security, bodycam	119-20-6	randaugment, hsv_h

(**a**) Three Pistols

(**b**) Robbery with Firearm



(**c**) Officer with Handgun



(**d**) People at Firing Range

Figure 4. Handgun images with Roboflow bounding boxes.

3.3. Performance Metrics

To validate our model's accuracy and reliability, we employed holdout validation, which involves training and testing the model on two separate data groups. While we considered using K-fold cross-validation, we found that it added complexity that stunted the dataset's compatability with Roboflow's features. We found that because of the class imbalance, K-fold poorly represented minority classes, and holdout validation was a more reliable way to assess our model's baseline performance. We used precision P, recall R, accuracy, and F1-scores directly from the ultralytics package. Our evaluation follows a binary classification approach, where positive instances represent detected objects and negative instances represent undetected ones. We use a confusion matrix, demonstrated in Table 3, for ease in comparing the model's detection accuracy with the actual presence of objects.

Table 3. Confusion matrix.

		True Value	
		Positive	**Negative**
Predicted Value	**Positive**	True Positive (TP)	False Positive (FP)
	Negative	False Negative (FN)	True Negative (TN)

The confusion matrix provides the data needed to calculate precision, recall, accuracy, and F1-scores.

Accuracy is the ratio of correctly classified instances by the classifier:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (6)$$

Error Rate: refers to the ratio of misclassified instances by the classifier:

$$Error\ Rate = \frac{FP + FN}{Total\ (TP + FP + FN + TN)} \quad (7)$$

Sensitivity (recall for true-positive rate (ground truth)):

$$Sensitivity = \frac{TP}{TP + FN} = \frac{Gun}{All\ Gun} = Recall \quad (8)$$

Specificity (true-negative rate):

$$Specificity = \frac{TN}{TN + FP} = \frac{No\ Gun}{All\ No\ Gun} \quad (9)$$

Precision: all instances detected as guns:

$$Precision = True\ Positive\ /\ (True\ Positive + False\ Positive) \quad (10)$$

Recall: precision of instances with guns:

$$Recall = True\ Positive\ /(True\ Positive + False\ Negative) \quad (11)$$

F1-score:

$$\begin{aligned} F_1\ Score &= \frac{2}{recall^{-1} + precision^{-1}} = \left(\frac{2 \times precision \times recall}{precision + recall}\right) \\ &= \frac{2}{\frac{TP+FN}{TP} + \frac{TP+FP}{TP}} = \frac{2}{1 + \frac{FN}{TP} + 1 + \frac{FP}{TP}} \\ &= \frac{2}{2 + \frac{FN+FP}{TP}} = \frac{TP}{TP + \frac{(FN+FP)}{2}} \end{aligned} \tag{12}$$

Balanced accuracy:

$$Balanced\ ACC = \frac{Sensitivity + Specificity}{2} \tag{13}$$

Matthew's Correlation Coefficient:

$$MCC = \frac{(TP * TN - FP * FN)}{\sqrt{(TP + FP) * (TP + FN) * (TN + FP) * (TN + FN))}} \tag{14}$$

Intersection over union:

$$IoU = \frac{TP}{TP + FN + FP} = \frac{Target \cap Prediction}{Target \cup Prediction} \tag{15}$$

Area under curve estimation:

$$\text{AUC} = \int_{x=0}^{1} y(x)\,dx \approx \sum_{i=1}^{n-1} \frac{(x_{i+1} - x_i)(y_i + y_{i+1})}{2} \tag{16}$$

The Matthew's Correlation Coefficient assesses the performance of binary classification models to quantify the correlation between predicted and actual instances. Both precision and recall emphasize true positives. Precision measures the accuracy of positive predictions, and recall evaluates the models ability to retrieve real positive instances. Next, the F1-score is a harmonic mean of precision and recall. Finally, average precision (AP) summarizes the precision and recall across all confidence levels. Our model displays the mean average precision (mAP) at every training epoch to aid in evaluating target localization. To determine the accuracy of a bounding box towards the ground truth (actual class instance), the intersection over union (IoU) metric quantifies the overlap between the predicted and ground truth bounding boxes, with 0.5 being the cutoff between true positives and negatives (15) (Figure 5).

Although conventional metrics such as accuracy (6), balanced accuracy (13), and the Matthew's Correlation Coefficient (14) are commonly used for binary classification tasks, they rely heavily on the presence of true negatives. In our situation, the dataset is purposefully composed of majority gun instances to challenge our model's classification and localization ability under positive scenarios. As a result, true negatives are minimal or nonexistant, rendering these metrics misleading or obsolete.

Since our dataset is so heavily skewed towards true-positive instances, to address this issue, we report both conventional metrics and mAP@0.5, mAP@0.5-0.95, F1-score, and accuracy, which are standard in object detection to address class imbalances. The AUC is a summated estimation of the area under the precision–recall curve by finding a function who's integral can estimate the area (16). For estimations, we use Python package sklearn's ROC AUC calculator with the trapezoidal rule. Additionally, we will include an extra section for more conventional metrics that factor into the final results.

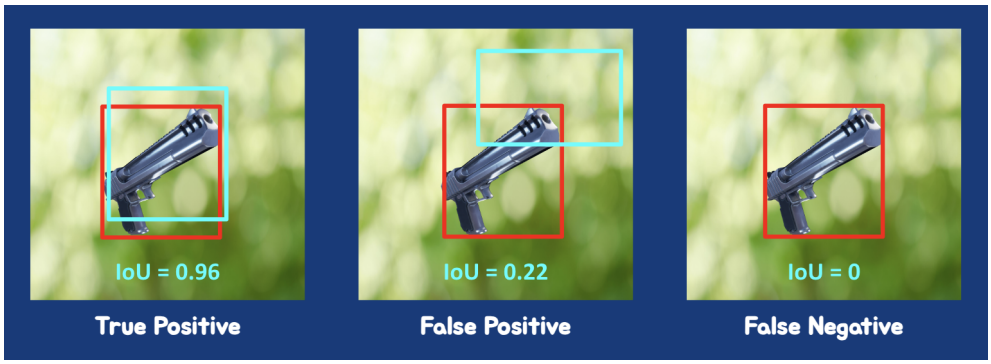


Figure 5. Intersection over union diagram.

3.4. Function and Hardware Specifications

For hardware, we used an A100 GPU and an RTX 4090 GPU from Tunghai University, using a Ubuntu 22.04 virtual machine to secure shell into the system [37]. The A100 was frequently used by students and computed operations incredibly quickly, almost double or triple the times of the 4090. However, we were able to use the 4090 used year-round, whereas the A100 was restricted to the summer months. Unfortunately, using two computers to train two different models benchmarked against each other raises discrepancies in training time and efficiency due to different hardware functionalities, and these need to be factored in to the final results. For full hardware details, refer to Table 4.

Table 4. Hardware specification.

Machine	Computation	Cores	RAM
Computer 1	NVIDIA A100 AMD 7302 @7.6GHz NVIDIA A100 40GB	32/64	216 GB
Computer 2	GeForce RTX 4090 AMD 7960Xs @3GHz NVIDIA RTX 4090	24/48	256 GB

4. Experimental Results

Our Experimental Results section discusses the training results for all YOLO models, including YOLOv5, YOLOv7, YOLOv8, YOLOv9, YOLOv10, and YOLOv11.

4.1. Training Results

In this subsection, we will discuss the process of training our models, the iterations we made to the training process, and the important metrics we studied.

4.1.1. Model Training Process

For our training process, we made sure to prepare a sufficient number of samples and implement appropriate augmentation techniques. From our split, around 15,000 samples were allocated for training, while the remaining 3000 were earmarked for validation and testing. Figure 6 illustrates the distribution of samples in all the datasets used in this study, which should only contain the singular class of handguns.

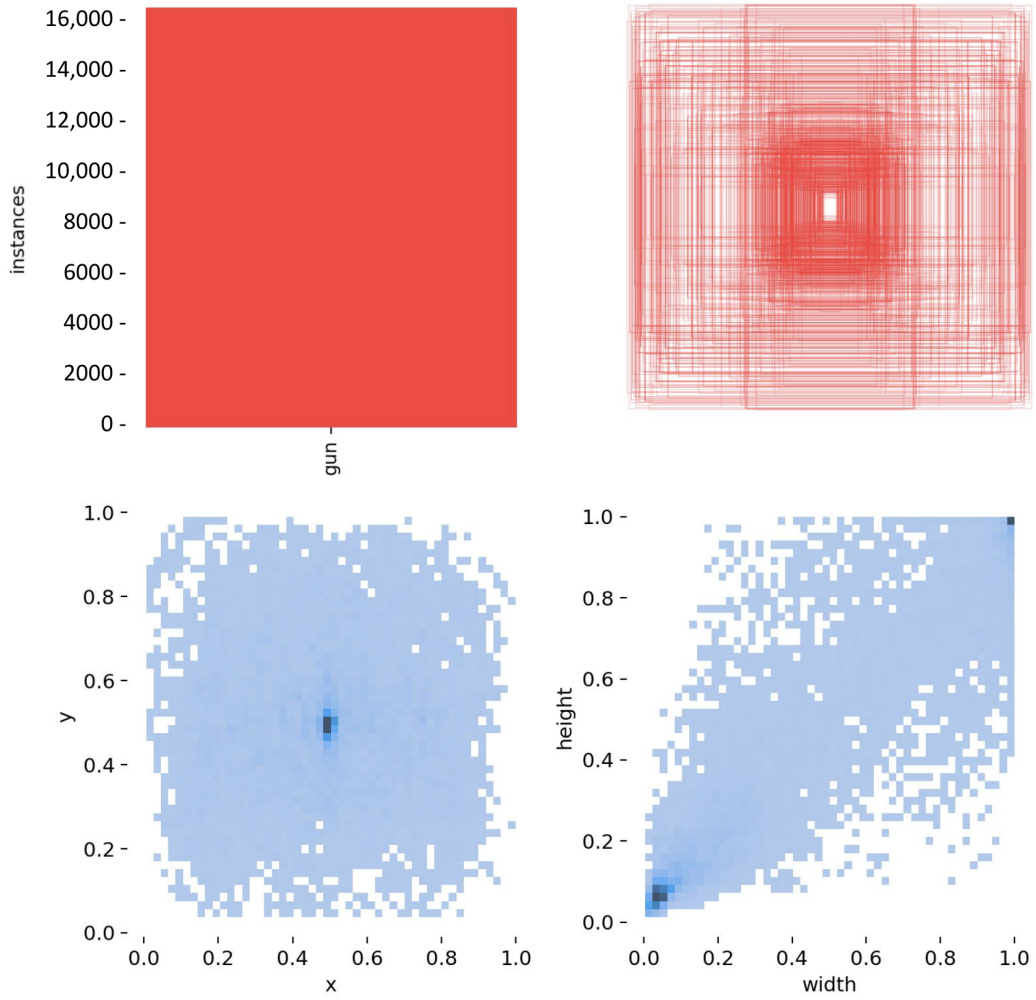


Figure 6. Labels by class.

These were some of the hyperparameters for our model:

- High Epochs and Low Patience: Epochs: 500–1200, which allowed the model to familiarize with training data. Patience: 100 allowed for a prolonged period of non-improving performance before triggering the early stoppage.
- Regularization: Weight_decay: 0.0005 applied L2 regularization to penalize large weighted neurons hogging up attention. As a countermeasure against overfitting, its strength lies in being fine-tuned.
- Data Augmentation: Our techniques of mosaic: 1.0 and auto_augment: randaugment helped prevent overfitting by artificially increasing the training dataset's diversity with altered examples of the same data, increasing the model's generalization ability.

4.1.2. Calculating Metrics

Because YOLOv8 is an anchor-free model, we choose to emphasize testing its individual versions as a control group, keeping a consistent high-detection speed. This approach eliminates challenges associated with anchor boxes, such as issues with differentiating box distributions across standard benchmarks. As an anchor-free model, YOLOv8 makes fewer predictions, leading to faster and more efficient performance, particularly enhancing the Non-Maximum Suppression process (Equation (5)). We trained YOLO versions 5s, 7s, 9s, 10s, and 11s for 500 epochs, noting diminished improvements beyond this range, while we trained each YOLOv8 task mode (nano, small, medium, large, and extra large)

for 800 epochs, tracking the holistic performance of YOLOv8 across different sizes. Our overall goal is to test weapon detection capabilities across different YOLO versions to get a holistic picture.

Our first observation was that our models struggled with small-object detection in live 360-degree security camera footage, such as in the Uvalde School shooting footage. Therefore, we adjusted the distribution focal loss (DFL) to 0.01 to focus more on key points than bounding boxes, a common solution to reduce sensitivity to intersection over union (IoU) (Equation (15)) discrepancies. Small-object detection can be optimized to improve future training sessions.

The metrics we mainly focus on can be found in Table 5. The training results for area under the curve, using Trapezoidal ROC-AUC summation, ranged from 0.7720 to 0.8310, indicating fair recall in differentiating positive classes from negative ones. In our case, we factored in the low amounts of true negatives to get the final result as typical AUC's tend to match mAP50 scores. YOLOv10 had the highest AUC, while YOLOv5 had the lowest.

For mAP0.5 and mAP0.5:0.95, the scores ranged from 0.9791 to 0.9838 for mAP0.5 and 0.7701 to 0.8720 for mAP0.5:0.95. This indicated excellent results in precision, demonstrating our model's ability to identify positive predictions accurately. This displays high class localization ability on our positively skewed dataset. YOLOv8x had the highest mAP0.5:0.95, and YOLOv7 had the lowest. YOLOv5 and YOLOv11 tied for the highest mAP0.5, and YOLOv10 had the lowest. Referencing our confusion matrix, every single model achieved above 90% identification of true-positive gun instances. Our confusion matrix can be found in Figure 7.

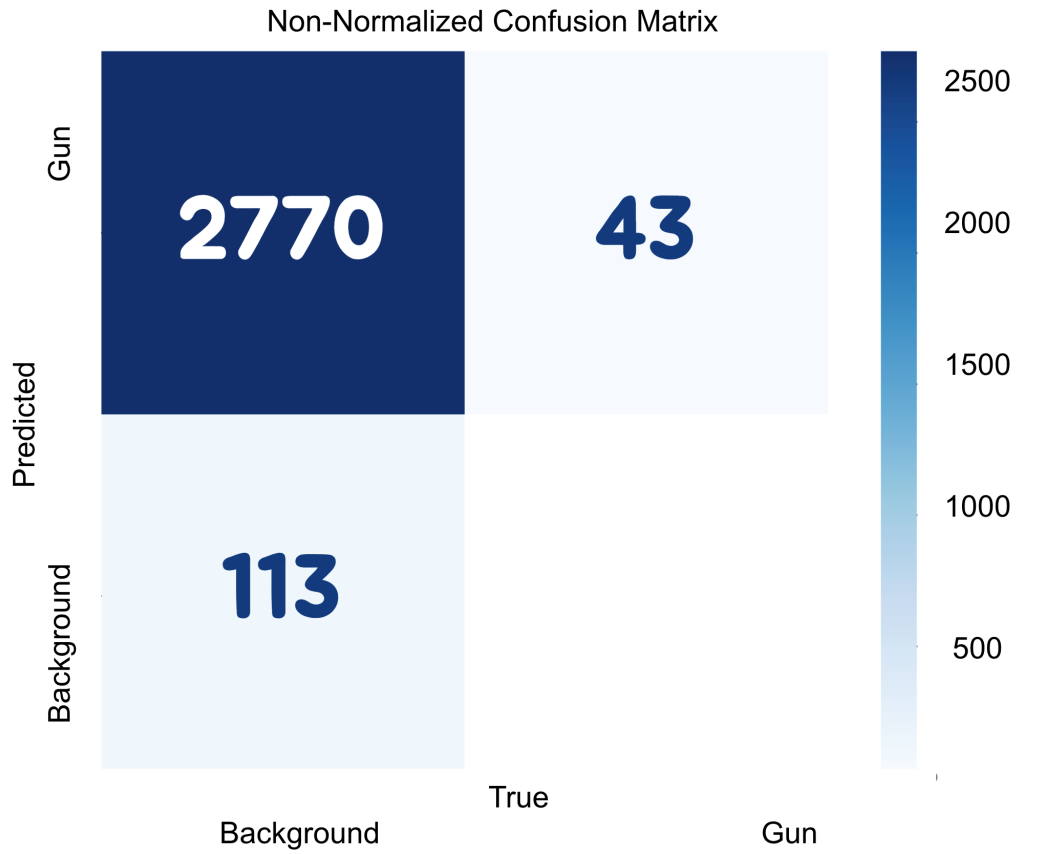


Figure 7. YOLOv8 confusion matrix.

Throughout our runtime process, we noticed that our models achieved different results at each epoch benchmark during training. Figure 8 illustrates our combined observations of our models' mAP0.5:0.95 values every 150 epochs. Some of them completed training

before others, and their stopping points are noted. Half of the models started at near 0 mAP, and the other half began around 0.5 mAP. By the 300th epoch, all the models had surpassed 0.7 mAP0.5:0.95. By the 600th, most of them surpassed 0.8. YOLOv8x, YOLOv8l, and YOLOv8m had the highest mAP values, while YOLOv7 had the lowest. Based on our results, we summarize that 300–600 epochs achieves the most consistent results.

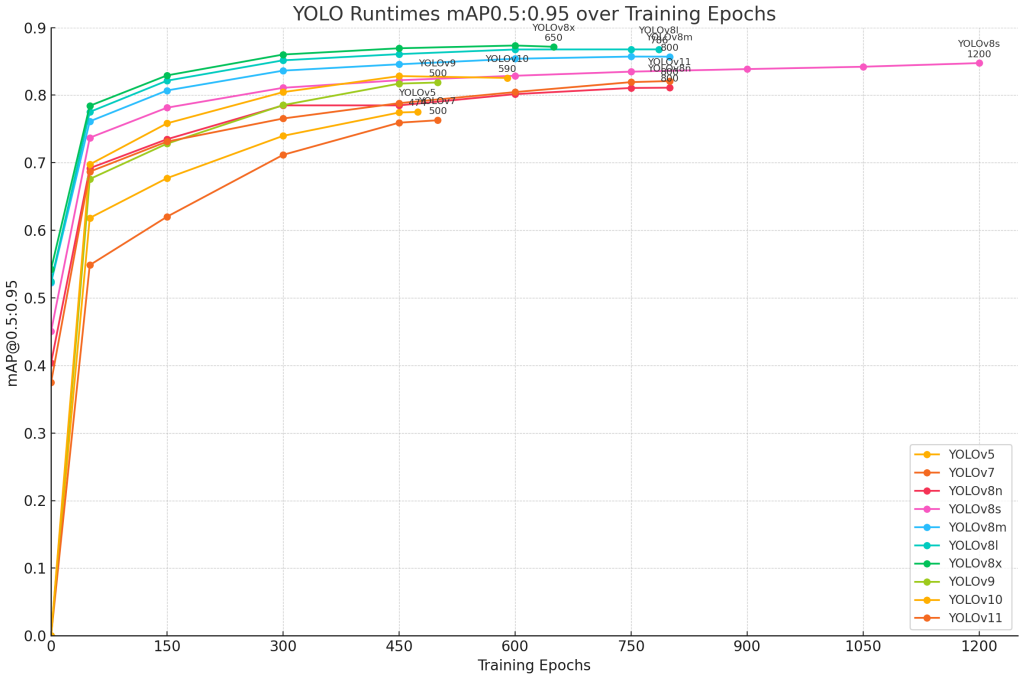


Figure 8. YOLO runtime comparisons.

Table 5. Performance comparison of YOLO model versions.

Model	F1-Score	Area Under Curve	mAP@0.5	mAP@0.5:0.95
Version 5	0.968	0.772	0.984	0.780
Version 7	0.968	0.811	0.983	0.770
Version 8n	0.969	0.831	0.983	0.812
Version 8s	0.969	0.821	0.981	0.848
Version 8m	0.970	0.784	0.981	0.857
Version 8l	0.967	0.831	0.981	0.868
Version 8x	0.968	0.821	0.981	0.872
Version 9	0.970	0.827	0.984	0.822
Version 10	0.958	0.831	0.979	0.825
Version 11	0.970	0.831	0.984	0.824

4.2. Training Aftermath

Because of the aforementioned skew in our dataset, conventional metrics gave a different perspective than the ones we focused on. This highlighted an important shortcoming in our testing process in the lack of high-quality true-negative instances, referring to Table 6. In our situation, a tradeoff to an increased handgun positive skew is a decrease in true-negative background receptivity, leading to potential "unfamiliarities" in non-handgun imagery. Our model is trained for high-precision task localization on positive datasets, which may render the following metrics misleading or obsolete.

Because of the black-box abstraction of certain YOLO designs, training results often lack the non-normalized confusion matrixes that allowed us to see the exact class counts. Therefore, we are unable to calculate meaningful accuracy metrics for certain runtimes. To summarize our findings, the accuracy (6) ranged from 0.936 to 0.955, demonstrating

well-above-average results due to the large high-precision positive classes. The balanced accuracy (13) ranged from 0.482 to 0.492, around half the accuracy. Our Matthew's Coefficient (14) ranged from -0.03 to -0.02, indicating a slight negative correlation. While these metrics paint the picture of our model performing inferior to textbook random chance, a potential shortcoming in precision-focused training, we loosely factor these metrics into the results due to our goal of purely maximizing precision that uniquely benefits security detection situations. For minor performance error analysis, the AI intervention should be prioritized as an assistive tool with human input to prevent potentially problematic false-alarm scenarios.

Table 6. Conventional performance metrics of YOLO models affected by dataset skew.

Model	Accuracy	Balanced Acc.	Matthew's Coef	Device	Runtime Hrs
Version 5	undef	undef	undef	4090	174
Version 7	undef	undef	undef	4090	22.4
Version 8n	0.942	0.484	−0.030	A100	8.34
Version 8s	0.9467	0.492	−0.025	A100	29.1
Version 8m	0.948	0.492	−0.025	A100	27.7
Version 8l	0.955	0.483	−0.020	A100	36.9
Version 8x	0.951	0.482	−0.022	4090	41.3
Version 9	undef	undef	undef	4090	20.3
Version 10	0.936	0.485	−0.033	4090	20.9
Version 11	0.946	0.490	−0.027	4090	201

For our training hardware, Table 6 also shows the computers and runtimes associated with each training. We had a batch size of 16, 82–14–4 split, and 500–800 epochs for each training. The models trained on the A100 achieved high average precision and greater results at lower epoch counts. After 150 epochs, the YOLOv8 variants all surpassed 0.7 mAP0.5:0.95. The models trained on the 4090 had moderate training results and took 300 epochs to surpase 0.7 mAP. The longest training time was YOLOv11 with 201 h, and this has been attributed to joint 4090 secure shell usage and YOLOv11 feature extraction enhancements that increase runtimes.

4.3. Comparing Image Results Between YOLO

After training, its important to obtain anecdotal evidence of the model's performance to promote reinforcement learning. What became immediately apparent was that almost all models excelled in real-life applications such as security camera footage, environments that blurred handheld objects, and large closeups, demonstrating general success against overfitting. Figure 9 shows our models performance on our validation set, specifically our YOLOv8x model. It succeeds in separating handgun instances in a single image with reasonably high confidence; however, it overfits on handgun instances at the detriment of rifle or submachine gun occurances.

Most models achieved tight bounding boxes around the 0.8–0.9 confidence intervals. For individual model differences, YOLOv7 stood out in its ability to separate different class instances in the same image, while YOLOv10 had great all-around performances and successfully limitted overfitting among different firearm types, even if our dataset emphasized handguns. Figure 10 shows our YOLOv10 model's performance on our dataset, and we consider its validation results to be some of the best out of our models. We theorize that this is due to the removal of non-maximum supression, which would normally eliminate excess bounding boxes that fail to meet confidence criteria, in replacement of a dual assignments that addresses potential class instances more intelligently.

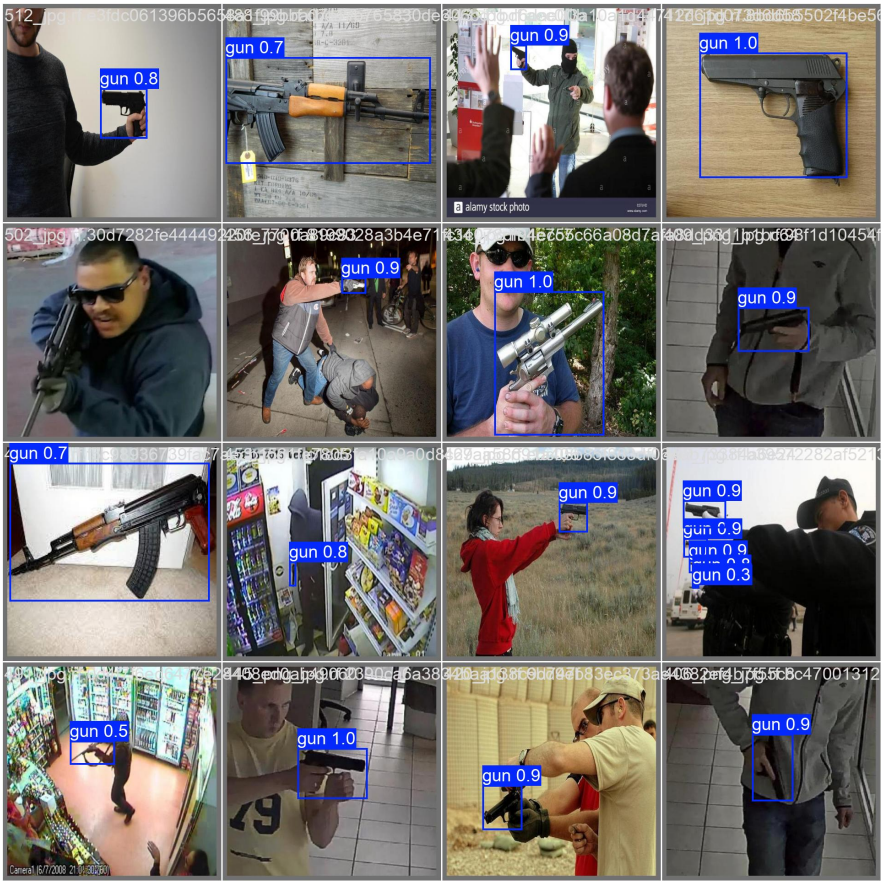


Figure 9. YOLOv8x validation set.

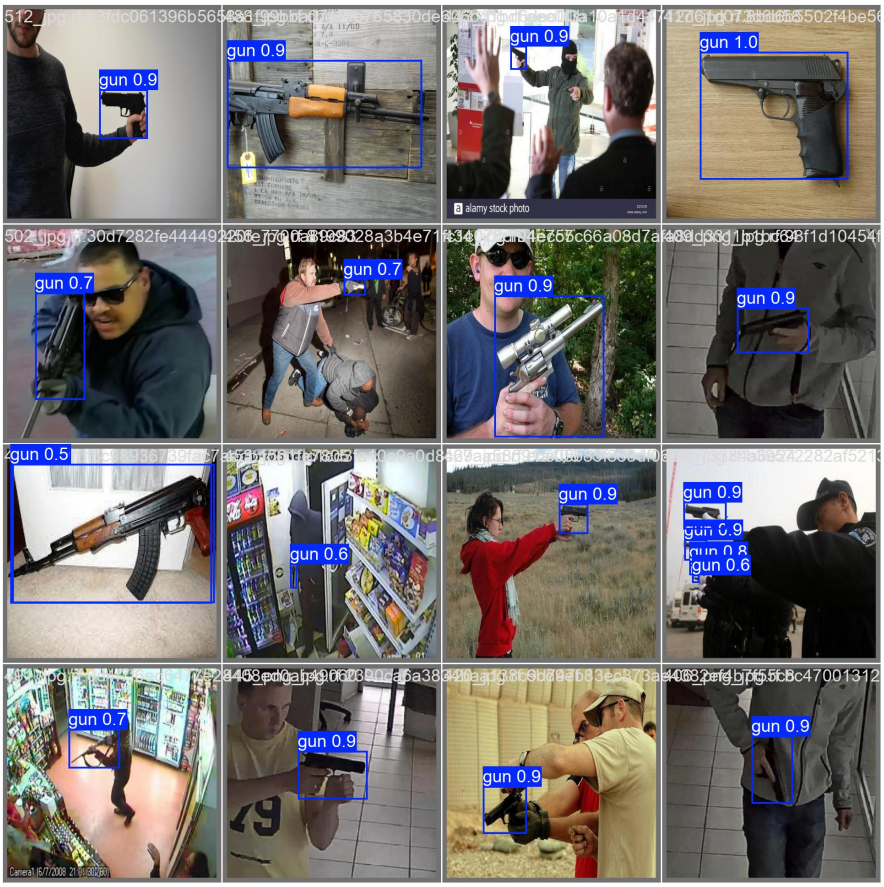


Figure 10. YOLOv10 validation set.

4.4. Testing Surveillance Capabilities

After training the data and evaluating the model weights, we implemented two testing mechanisms. The first involved using YOLO models to detect firearms in video footage, including mass shooting security cam recordings. As shown in Figure 11, the model struggled to detect rifles, particularly modified ones like the AR-15 used by the perpetrator. From a 360-degree angle, handguns appeared distorted, but YOLOv8s successfully detected one officer's handgun. Overall, the model demonstrated high accuracy in identifying handguns, but its performance with modified rifles and distorted objects was less reliable.

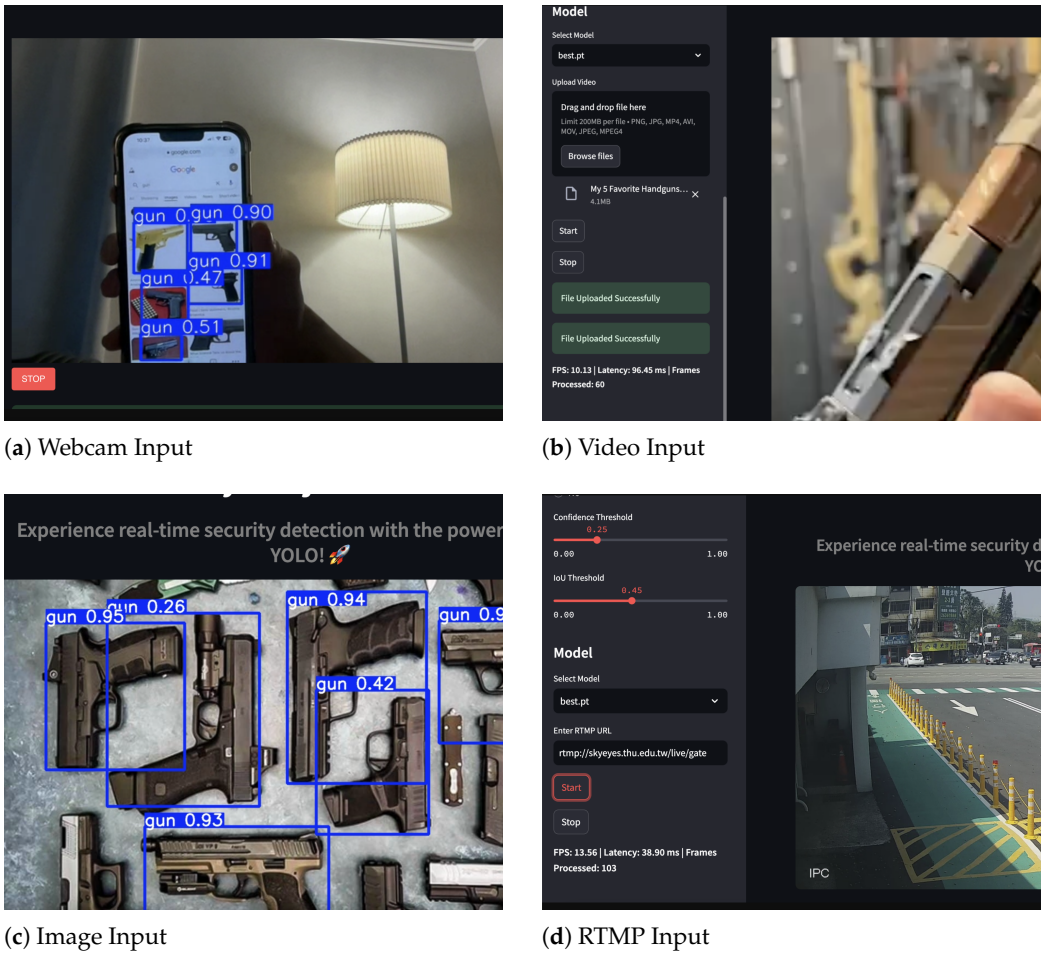
Figure 11. Tests on real-life mass shooting footage.

Edge Computing and General Detection Capabilities

Edge computing is a model that brings data processing closer to its source, reducing the reliance on distant cloud servers [38]. Examples include smart thermostats, wearable devices, and security systems, which often integrate AI to enhance functionality. In security settings, video surveillance machines are often required to run 24/7 to meet the desired needs [39]. Scaling AI tools can cause potential latency issues, upcharged costs for third-party API calls, and security vulnerabilities. Therefore, having a computer program that operates directly on a piece of sensing equipment or a local hosting platform can address these issues and create new innovative opportunities.

We set out to create an open-source hosting script with an adaptable setup that fits multiple settings, including homes and large businesses. We tested a Python version 3.11 based frontend using Streamlit version 1.32, a popular library for building web apps with machine learning capabilities. Streamlit's key advantage lies in its seamless integration between machine learning models and web outputs. Figure 12 contains images of our website's design and functionalities.

We optimized our project design for efficiency, adaptability, and ease of use. With features such as webcam video, RTMP streaming, and file uploads, we streamline the process of connecting artificial intelligence models to live streams and necessary services. The Real-Time Messaging Protocol connects users to live streams across the internet, enabling accessible security footage for object detection purposes. With built-in features that calculate useful metrics for our live streams, we enable efficient benchmarking of different models for security purposes. These features include the active framerate and latency-per-frame. Lastly, this open-source application is available on multiple devices and the Python web provider Streamlit, allowing for a diverse set of use cases such as edge devices and open access.



(**a**) Webcam Input (**b**) Video Input

(**c**) Image Input (**d**) RTMP Input

Figure 12. AI-integrated website through streamlit.

Refer to Table 7 for our website's latency data. We tested the application on an M3 Max Macbook Pro, an RTX 4090 GPU, an RTX A100 GPU, and our public streamlit application. The local hosted edge devices outperformed the website in terms of latency and FPS. Both GPU's achieved greater than 15 FPS on all live-detection applications. Our video and image processing latency ranged 8.6–57.34 ms, displaying reasonable results and no singular pattern across machinery. The streamlit web application ranged from 127 to 2648 milliseconds. The frames per second ranged from 5 to 300, demonstrating limitations but also significant potential for this technology. To summarize, our website is a simple application of object detection optimized for consistent usage and performance benchmarking.

Table 7. Hosting latency times on different devices.

Hosting Device	IMG ms	VID ms	WEB fps	VID fps	RTMP fps
M3 Max Macbook	49.29	51.33	7.53	20.05	15.4
RTX 4090 GPU	26.38	3.89	26.23	293.01	28.26
RTX A100 GPU	57.34	8.60	16.96	109.27	14.86
Streamlit Web App	2648.47	127.13	5.97	7.78	5.78

4.5. Discussions

This discussion section will discuss findings, limitations, tie-in-the results, and the current state of the literature.

We incorporated a skew in our dataset, targeting over 99% positive instances in handguns. Our hypothesized benefits were high-class localization on gun instances, while the potential downsides included true-negative hallucinations and background discernment troubles. After training analysis, we found that our models achieved amazing precision across the 50 and 95 confidence intervals. Therefore, we justify that our high-precision design directly benefits from our skewed dataset due to the class localization focus. For security instances where speed and reliability take emphasis over meticulous accuracy, models great at identifying true positives can adequately support security personnel, giving warnings that help people better identify the threat. As it is important to discuss potential limitations of this strategy, our lower balanced accuracy and Matthew's Correlation Coefficient paint a potential picture of our model struggling with discerning background instances. In our case, we strike a balance between prioritizing our high-precision and giving attention to our low conventional metrics.

Potential limitations of our model included holdout validation and our sole usage of handguns. More advanced splits and different target classes are utilized to prevent overfitting in weapon type and environment settings. While some of our models displayed examples of overfitting, our YOLOv8x and YOLOv10 models demonstrated great training against overfitting. Different weapon types were discerned successfully, and confidence levels remained high among diverse augmentations. Satisfied with these results, we chose to stick with the more computationally efficient holdout validation, as opposed to cross-validation, which was less compatible with our dataset. Looking ahead, our future work may include cross-validation.

Our web Python script was a small step towards fully integrating our model into an edge computing system. YOLO's lightweight design means that most systems can run detections with $\geq$10 FPS reliably. Our design is optimized for efficiency and adaptability, enabling easy benchmarking of different models against each other. While not fully Internet of Things (IoT), we plan to connect diverse data sources—such as body cams, emergency calls, and gunpowder sensors—into a cohesive detection system. This would improve threat detection accuracy and reduce false alarms. To fine-tune the model with reinforcement learning, we propose future pipelines that could connect possible detections to experts for verification, developing the model for its specific required environment.

To address some of the guidelines on AI reporting in our literature, we acknowledge that our dataset uses United States police body camera footage. This might skew detection results towards people groups most affected by police activity in the United States or misrepresent needs in other places of the world. We strategize mitigating bias through designing diverse datasets suitable in various environments and places. Next, our models and Python website are freely available for reproduction under a Creative Commons license. We use appropriate metrics and statistical analysis tools such as mAP50 and accuracy. For ethical considerations, our dataset uses information collected from security and body camera footage. While this raises right-to-privacy issues, individuals with unlawful weapons caught on security camera footage often become threats to public safety, where police can lawfully detain the suspects. Our model acts as an assistive tool to security personnel for human–AI interaction. For version control, we recommend using our YOLOv10 pretrained model for reproducibility.

The current state of the firearm detection literature can be broken down into dataset design and precision optimization [40]. Researchers have been primarily choosing images based on actual events or specific weapon instances [41]. Our dataset is comprised of publically available police body camera footage and video surveillance photos. While we were not able to decrease the number of false positives drastically, we were able to raise the precision to nearly 100%. Next, some authors experiment with switching elements of

the YOLO architecture including backbones and detection heads [42,43]. Many authors found trade-offs between performance metrics and speed, which influenced our decision to focus on utilizing the YOLO base model [44]. Future work could involve design changes to promote detection speed, with the goal of retaining the high-precision metric. To summarize, our model was designed to best optimize on past research while leaving room for additional changes.

5. Conclusions and Future Work

This paper demonstrates the reliability of YOLO object detection models for weapon detection, showing that YOLO can be effectively trained and deployed in video surveillance environments with high efficiency and speed. Our novel contributions include our precision-focused design that optimizes precision while effectively preventing overfitting in some of our models. It is useful in gun detection scenarios. Additionally, we introduce a website script that can connect object detection to various devices and use cases, allowing easy access to valuable technology. We recommend using later YOLO versions that are best suited to the deployment environment. For our model, while accuracy was slightly impacted by false positives, it performed excellently in true-positive detections and showed robustness against interference. By leveraging deep learning and labeled data from platforms like Roboflow, we achieved accurate firearm detection at various distances. Unlike manually operated security cameras, the model avoids issues like screen fatigue and poor eye health. YOLO's fast real-time performance is ideal for security applications, and we plan to incorporate notification systems, such as email or iMessage alerts, for timely responses. In areas with legal gun ownership, alert-free zones could be established, and police could benefit from improved communication in active shooting situations. This technology, particularly when trained on powerful GPUs like the NVIDIA A100, could be life-saving by providing continuous, automated updates to law enforcement. Future projects with Tunghai University will further explore the use of these advanced machine learning models.

Author Contributions: J.H. is a first-year student at the University of California, Santa Barbara, studying Computer Science for his B.S. degree. He is the main author. C.-T.Y. is Lifetime Professor of Computer Science at Tunghai University in Taiwan. He is the corresponding author. Conceptualization, J.H.; Methodology, J.H. and C.-T.Y.; Software, J.H.; Validation, J.H. and C.-T.Y.; Formal analysis, J.H.; Investigation, J.H.; Resources, C.-T.Y.; Writing–original draft, J.H.; Writing–review and editing, J.H. and C.-T.Y.; Visualization, J.H.; Supervision, C.-T.Y.; Project Administration, J.H.; Funding Acquisition, C.-T.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This work was sponsored by the National Science and Technology Council (NSTC), Taiwan, under Grant Nos. 114-2221-E-029-025-MY3; 114-2622-E-029-003; 113-2221-E-029-028-MY3.

Data Availability Statement: All our results can be found on our GitHub. Our image results, training data, and models can be found at https://github.com/Jonathan-Hsueh/Gun-Detection-Models (accessed on 21 August 2024). The object detection website to test the speed metrics can be found at https://github.com/Jonathan-Hsueh/YOLO-Object-Detection-Website/tree/main (accessed on 21 August 2024). All of this is free to use and open source.

Acknowledgments: We sincerely thank the anonymous reviewers for their valuable comments on this paper, which have allowed us to improve and polish its contents.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

YOLO	You Only Look Once
SSD	Single-Shot Detector
RCNN	Region-Based Convolutional Neural Network
SOFM	Self-Organizing Feature Map
BMU	Best-Matching Unit

References

1. Smart, R.; Schell, T.L. Mass shootings in the United States. In *Contemporary Issues in Gun Policy: Essays from the RAND Gun Policy in America Project*; RAND: Santa Monica, CA, USA, 2021; Volume 1, pp. 1–25.
2. Flynn, C.; Heitzmann, D. Tragedy at Virginia Tech: Trauma and Its Aftermath. *Couns. Psychol.* **2008**, *36*, 479–489. [CrossRef]
3. Anisin, A. Case Study Analysis of CEE Mass Shootings. In *Mass Shootings in Central and Eastern Europe*; Palgrave Macmillan: Cham, Switzerland, 2022; pp. 85–106.
4. Jehanzaib; Bangash, A.K. Students' satisfaction from the security measures taken in bacha khan university, charsadda after the terrorist attack. *Pak. J. Soc. Educ. Lang.* **2019**, *5*, 29–41.
5. Li, Y. Research and application of deep learning in image recognition. In Proceedings of the 2022 IEEE 2nd International Conference on Power, Electronics and Computer Applications (ICPECA), Shenyang, China, 21–23 January 2022; pp. 994–999.
6. Uetama, T.R.; Setiawan, W.; Sofyan, E. Performance comparation of real time image processing face recognition for security system. In Proceedings of the Conference on Management and Engineering in Industry, Marina Bay Sands, Singapore, 14–17 December 2020; Volume 2, pp. 21–25.
7. Baharloo, M.; Aligholipour, R.; Abdollahi, M.; Khonsari, A. ChangeSUB: A power efficient multiple network-on-chip architecture. *Comput. Electr. Eng.* **2020**, *83*, 106578. [CrossRef]
8. Islam, M.R.; Ahmed, M.U.; Barua, S.; Begum, S. A systematic review of explainable artificial intelligence in terms of different application domains and tasks. *Appl. Sci.* **2022**, *12*, 1353. [CrossRef]
9. Jain, H.; Vikram, A.; Kashyap, A.; Jain, A. Weapon detection using artificial intelligence and deep learning for security applications. In Proceedings of the 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 2–4 July 2020; pp. 193–198.
10. Noor Afandi, W.E.I.B.W.; Isa, N.M. Object Detection: Harmful Weapons Detection using YOLOv4. In Proceedings of the 2021 IEEE Symposium on Wireless Technology & Applications (ISWTA), Virtually, 17 August 2021; pp. 63–70.
11. Ahmed, A.A.; Echi, M. Hawk-eye: An AI-powered threat detector for intelligent surveillance cameras. *IEEE Access* **2021**, *9*, 63283–63293. [CrossRef]
12. Jiang, P.; Ergu, D.; Liu, F.; Cai, Y.; Ma, B. A Review of Yolo algorithm developments. *Procedia Comput. Sci.* **2022**, *199*, 1066–1073. [CrossRef]
13. Kiran, A.; Purushotham, P.; Priya, D.D. Weapon Detection using Artificial Intelligence and Deep Learning for Security Applications. In Proceedings of the 2022 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), Bhubaneswar, India, 19–20 November 2022; pp. 1–5.
14. Tabassum, H.; Najmusher, H.; Anjum, A.; Jenita, J.; Humera, D.; Viji, C. Weapon Detection System for the Prevention of any Potential Crime Using Artificial Intelligence. In Proceedings of the 2023 5th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 15–16 December 2023; pp. 1067–1072.
15. Xu, S.; Hung, K. Development of an AI-based system for automatic detection and recognition of weapons in surveillance videos. In Proceedings of the 2020 IEEE 10th Symposium on Computer Applications & Industrial Electronics (ISCAIE), Penang, Malaysia, 18–19 April 2020; pp. 48–52.
16. Aung, Y.Y.; Oo, K.Z. Detection of Guns and Knives Images Based on YOLO v7. In Proceedings of the 2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT), Vellore, India, 3–4 May 2024; pp. 1–6.
17. Deshpande, D.; Jain, M.; Jajoo, A.; Kadam, D.; Kadam, H.; Kashyap, A. Next-Gen Security: YOLOv8 for Real-Time Weapon Detection. In Proceedings of the 2023 7th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), Kirtipur, Nepal, 11–13 October 2023; pp. 1055–1060.
18. Jadhav, V.; Deshmukh, R.; Gupta, P.; Ghogale, S.; Bodireddy, M. Weapon Detection from Surveillance Footage in Real-Time Using Deep Learning. In Proceedings of the 2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA), Pune, India, 18–19 August 2023; pp. 1–6.

19. Borthakur, S.; Kumar, G.; Rajput, A.; Sarvaiya, J.N. Object Detection for Military Surveillance using YOLO Framework. In Proceedings of the 2023 IEEE 20th India Council International Conference (INDICON), Hanamkonda, India, 14–17 December 2023; pp. 126–131.
20. Hao, X.; Yang, S.; Liu, R.; Feng, Z.; Peng, T.; Huang, B. VSLM: Virtual Signal Large Model for Few-Shot Wideband Signal Detection and Recognition. *IEEE Trans. Wirel. Commun.* **2025**, *24*, 909–925. [CrossRef]
21. Kumar, A.; Zhang, Z.J.; Lyu, H. Object detection in real time based on improved single shot multi-box detector algorithm. *EURASIP J. Wirel. Commun. Netw.* **2020**, *2020*, 204. [CrossRef]
22. Gupta, G.; Chattopadhyay, S.; Kukreja, V.; Aeri, M.; Mehta, S. The Arsenal Algorithm: AI-Driven Weapon Recognition with CNN-SVM Model. In Proceedings of the 2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 14–15 March 2024; pp. 1–6.
23. Hnoohom, N.; Chotivatunyu, P.; Maitrichit, N.; Sornlertlamvanich, V.; Mekruksavanich, S.; Jitpattanakul, A. Weapon Detection Using Faster R-CNN Inception-V2 for a CCTV Surveillance System. In Proceedings of the 2021 25th International Computer Science and Engineering Conference (ICSEC), Chiang Rai, Thailand, 18–20 November 2021; pp. 400–405.
24. ultralytics. GitHub—ultralytics/yolov5: YOLOv5 in PyTorch > ONNX > CoreML > TFLite. Available online: https://github.com/ultralytics/yolov5 (accessed on 11 August 2024).
25. WongKinYiu. GitHub—WongKinYiu/yolov7: Implementation of Paper—YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. Available online: https://github.com/WongKinYiu/yolov7 (accessed on 11 August 2024).
26. ultralytics. GitHub—ultralytics/ultralytics: NEW—YOLOv8 in PyTorch > ONNX > OpenVINO > CoreML > TFLite. Available online: https://github.com/ultralytics/ultralytics (accessed on 11 August 2024).
27. WongKinYiu. GitHub—WongKinYiu/yolov9: Implementation of Paper—YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. Available online: https://github.com/WongKinYiu/yolov9 (accessed on 11 August 2024).
28. THU-MIG. GitHub—THU-MIG/yolov10: YOLOv10: Real-Time End-to-End Object Detection. Available online: https://github.com/THU-MIG/yolov10 (accessed on 11 August 2024).
29. Ultralytics. YOLO11 NEW. Available online: https://docs.ultralytics.com/models/yolo11/ (accessed on 16 December 2024).
30. Redmon, J. YOLO: Real-Time Object Detection. Available online: https://pjreddie.com/yolo/ (accessed on 11 August 2024).
31. Hyams, G.; Malowany, D. The Battle of Speed vs. Accuracy: Single-Shot vs Two-Shot Detection Meta-Architecture. Available online: https://clear.ml/blog/the-battle-of-speed-accuracy-single-shot-vs-two-shot-detection (accessed on 11 August 2024).
32. Shroff, M. Know Your Neural Network Architecture More by Understanding These Terms. Available online: https://medium.com/@shroffmegha6695/know-your-neural-network-architecture-more-by-understanding-these-terms-67faf4ea0efb (accessed on 18 December 2024).
33. Roboflow. Roboflow: Computer Vision Tools for Developers and Enterprises. Available online: https://roboflow.com/ (accessed on 11 August 2024).
34. Dana. CS 231N Project Dataset. Roboflow Universe 2024. Available online: https://universe.roboflow.com/dana-q9plh/cs-231n-project (accessed on 11 August 2024).
35. Ahmed, S.; Bhatti, M.T.; Khan, M.G.; Lövström, B.; Shahid, M. Development and optimization of deep learning models for weapon detection in surveillance videos. *Appl. Sci.* **2022**, *12*, 5772. [CrossRef]
36. Dobbin, K.K.; Simon, R.M. Optimally splitting cases for training and testing high dimensional classifiers. *BMC Med. Genom.* **2011**, *4*, 31. [CrossRef] [PubMed]
37. Ubuntu. Available online: https://ubuntu.com/download/desktop (accessed on 11 August 2024).
38. Singh, A.; Anand, T.; Sharma, S.; Singh, P. IoT based weapons detection system for surveillance and security using YOLOV4. In Proceedings of the 2021 6th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 8–10 July 2021; pp. 488–493.
39. Hashmi, T.S.S.; Haq, N.U.; Fraz, M.M.; Shahzad, M. Application of deep learning for weapons detection in surveillance videos. In Proceedings of the 2021 International Conference on Digital Futures and Transformative Technologies (ICoDT2), Islamabad, Pakistan, 20–21 May 2021; pp. 1–6.
40. Brahmaiah, M.; Madala, S.R.; Chowdary, C.M. Artificial intelligence and deep learning for weapon identification in security systems. *J. Physics Conf. Ser.* **2021**, *2089*, 012079. [CrossRef]
41. Hashim, N.; Anto Sahaya Dhas, D.; Jayesh George, M. Weapon detection using ML for PPA. In Proceedings of the Third International Conference on Intelligent Computing, Information and Control Systems: ICICCS 2021, Madurai, India, 6–8 May 2021; pp. 827–841.
42. Bhatti, M.T.; Khan, M.G.; Aslam, M.; Fiaz, M.J. Weapon detection in real-time CCTV videos using deep learning. *IEEE Access* **2021**, *9*, 34366–34382. [CrossRef]

43. Wang, G.; Ding, H.; Duan, M.; Pu, Y.; Yang, Z.; Li, H. Fighting against terrorism: A real-time CCTV autonomous weapons detection based on improved YOLO v4. *Digit. Signal Process.* **2023**, *132*, 103790. [CrossRef]
44. Santos, T.; Oliveira, H.; Cunha, A. Systematic review on weapon detection in surveillance footage through deep learning. *Comput. Sci. Rev.* **2024**, *51*, 100612. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.